



Vilnius
University

MINI PROJECT

INTERNATIONAL
CYBERSECURITY
AND
CYBERINTELLIGENCE

2025



VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
CYBERSECURITY LABORATORY

International Cybersecurity and Cyberintelligence 1st year Research Project

NAS : Next.js Attack Surface

Done by:

Giovanni Menon

Raphael Toubol

Christopher Samson Nyandoro

Supervisor:

prof. dr. Agnė Brilingaitė

Vilnius
2025

Contents

Introduction	4
1 Related Work	5
2 Methodology	6
2.1 Requirements	6
2.2 Tool Design and Architecture	7
2.2.1 System Context	7
2.2.2 System Architecture	7
2.2.3 Iterative Design	8
2.2.4 Implementation Overview	9
2.2.5 Component Details	9
3 Results	11
3.1 Vulnerable Next.js Project Analysis	11
3.2 Community Next.js Project Analysis	11
4 Discussion	11
4.1 Experiment Evaluation	12
4.2 Limitations	13
Conclusions and Future Work	14
References	15

Introduction

Web Application development and tooling has transformed drastically since the introduction and widespread adoption of single-page applications and full-stack JavaScript frameworks. This modern web application development architecture provides significant advantages in terms of tooling, user and developer experience, however introduces increased complexity in the realm of security.

Necessity for complex security verification comes from the implications of insecure web applications in the modern age. This coupled with the multi-faceted attack surfaces in web applications dictates an essential requirement for heightened security posturing from developers along with adequate tooling for quickly and effectively completing attack-surface analysis.

Due to the high volume and criticality of vulnerabilities in web-app specific code and the plethora of vulnerabilities the javascript third-party dependency libraries this is a focus area of this scientific research.

Therefore, the **aim** of the work is to provide a Golang config file based tool specifically for the purpose of discovering and surfacing vulnerabilities in API Routes and dependencies in React based Next.js full stack applications directly from their source code repositories. To achieve the aim the following tasks are formulated:

- analyze related scientific work and tools already present;
- modeling and developing a specific tool that satisfies the requirements;
- implement and evaluate our solutions.

From here after the tool will be referred to as NAS (Next-Attack-Surface). Attack surface in the context of the NAS project is:

- known vulnerable NPM packages
- unsafe patterns ignoring React/NextJS safe best practices *or patterns that deliberately nullify safety features present in NextJS & React*

The document is structured as follows. Section 1 provides the current landscape of our work, while Section 2 describes in detail the logic and requisites of our implementation. In Section 3 we present the solution and the experimental setup. Section 4 is devoted to the discussion of the obtained results. Section 4.2 summarizes the work and provides possible future directions based on our study.

1. Related Work

This section outlines and relates the scientific work of this paper to the foundational academic literature it is built upon. In his study, Hellquist has scientifically investigated and academically discussed the security of JavaScript front-end frameworks. The author discusses, compares, and contrasts the security features and inherent vulnerabilities of several popular front-end frameworks including React, Vue & Svelte. The Next.js framework extends react from the client to the server, which is why the underlying security features and vulnerabilities of react are so important [5].

A considerable and documented vulnerability source in modern web applications comes directly from developers leveraging third-party dependencies. Lauinger et al. in their study analyses different data from over 133,000 finding 37% to have outdated libraries. With a lead time between vulnerability discovery and patching to be over a year the industry cannot rely on automatically upgrading packages and needs to leverage vulnerability scanning tools [6].

Decan et al. analyses and discusses about the Node Package Manager (NPM) effect in the Javascript landscape. This propagation of vulnerability throughout the JavaScript ecosystem occurs as a result of the widespread and high availability of outdated and vulnerable libraries. This dependency relationship and vulnerability reality can occur via transitivity when the developer leveraging a package inherits a vulnerability from a third-party dependency several libraries upstream of the implemented library in the web application codebase [3].

Specifically, the research presented by Anastasia and Stamatia discuss possible target mitigation strategies for react specific dependency vulnerabilities [1].

In specific cases, the identification of vulnerabilities requires automated static analysis. These static analysis tools analyze code without execution, Brito et al. present a complete comparison of the capabilities of several tools geared toward the Node.js runtime. This highlights the power of static analysis while also demonstrating some of the shortcomings in the current available solutions [2].

The research discussed and thoroughly documents the probable security issues in Next.js applications inherited from the wider JavaScript, NPM, and React ecosystems. The existence of static analysis tools is also outlined; however, there is an evident need for easily accessible tooling that consolidates the capabilities of effective vulnerability detection tools into one quick and easy developer-focused tool. Consolidated tooling removes the burden on the developer to implement and use a wide variety of tools to simply confirm the safety of their application. Therefore the objective of this research is to provide a solution consolidating existing functionalities from disparate tools one consolidated CLI based system with ease and speed of use to encourage adoption and mitigation of active vulnerabilities in the javascript ecosystem.

2. Methodology

This section describes in detail the logic and requisites of our implementation, showing the underlying logic and the procedures used to carry out our product. Section 2.1 describes the requisites behind our work, while Section 2.2 is devoted to a description of our product.

2.1. Requirements

The main goal of this tool was to implement and consolidate attack surface analysis tooling from multiple other tools into one simple and quick interface. To adequately address the needs of developers, and complete a well rounded attack surface analysis there are several key functional requirements which need to be addressed:

1. Scanning of package dependencies (and dev-dependencies) for vulnerabilities
This is a core requirement of this piece of work due to our analysis of the currently published works [3, 6, 1].
2. Static application security testing (SAST) scanning of code for:
 - Clear-text passwords
 - Exposed CORS attack surface
 - Remote code execution
 - API keys
 - Sensitive data in comments & cleartext
 - Sensitive data in CORS
3. associating identified vulnerable packages with CVE numbers
4. surfacing the information in a visual format
5. configuration via file

Meeting these key functional requirements would equate to a tool that could be leveraged by all Next.js developers for dependency scanning, static analysis & identification of information leakage through external api's. It is this projects goal to not only meet these requirements but exceed them.

Several non-functional requirements were also identified:

- **Graceful handling of corrupt files:** Any corrupt files in the repository being analyzed should not crash the program
- **No source code middle-maning:** With a focus on security it is important that the source code being analyzed is not transmitted to a third party or to any external database
- **Modular Architecture:** Uncoupled or loosely coupled feature architecture addition of future functionality will be easier
- **Logging:** Comprehensive logging should be available to any user by specifying a verbose flag

2.2. Tool Design and Architecture

2.2.1. System Context

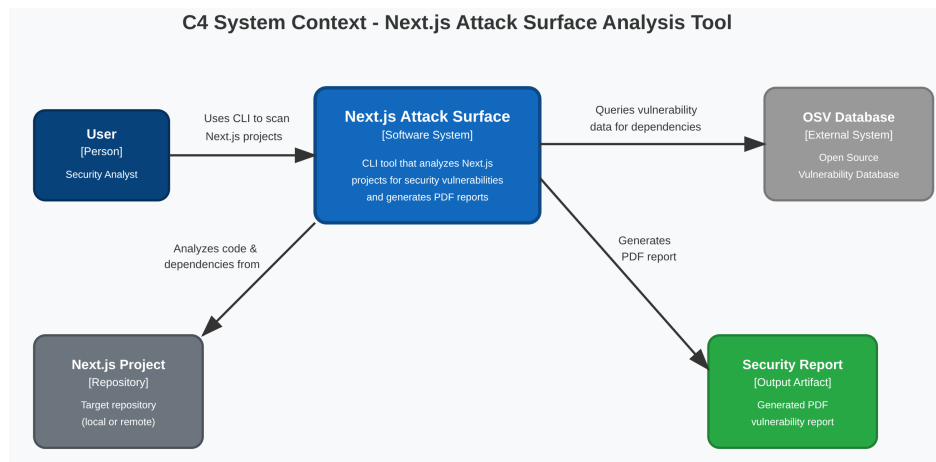


Figure 1: NAS System Context

As depicted in 1 the System context involves a user initializing the CLI with a config file or arguments, the Next.js attack surface software system then analyzing code and dependencies from the Next.js project repository, Querying the OSV Database for vulnerabilities and ultimately producing an output artifact - the security report.

2.2.2. System Architecture

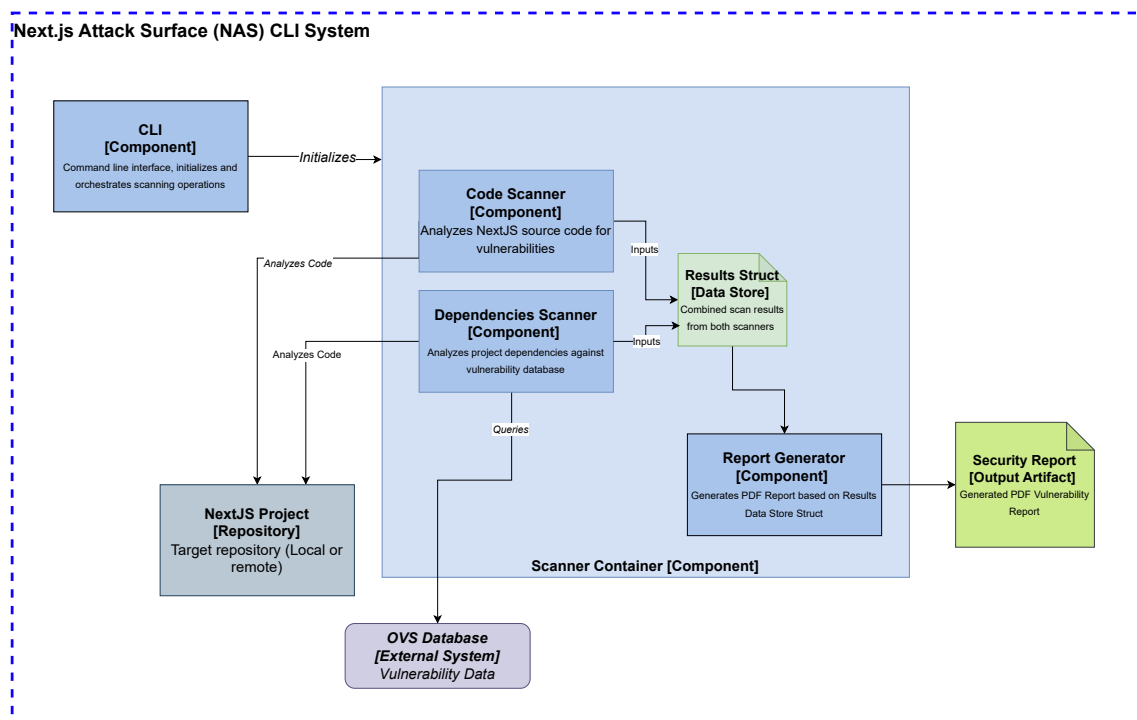


Figure 2: NAS System Architecture

As depicted in Figure 2, NAS proposes a modular design with several components. These components being:

- **Command Line Interface:** Initialization, Orchestration and user input
- **Code Scanner:** Statically Analyzes Next.js source code for vulnerabilities
- **Dependency Scanner:** Analyzes project dependencies
- **Report Generator:** Generates a human readable & professional report of the findings

The NAS Tool also has several other elements depicted in the system architecture:

- **Next.js Project:** The source repo, the subject of the code analysis
- **OVS Database:** The external source for vulnerability data
- **Results Struct:** The data structure which the components inside the scanner container component share for data storage
- **Security Report:** This is the NAS tools output artifact, the professional and human readable report of findings

2.2.3. Iterative Design

For this project an iterative design approach was adopted, with the fail fast mentality due to the time constraints and scope size. The system architecture depicted in 2 is the final conceptual design. Throughout the spiking iterations many modifications were made to the design approach. These include:

1. Partial implementation of the dependency scanner with no external system dependency. Initially the dependency scanner had no external dependency for a vulnerability database. Vulnerabilities were imported from external database api's into a local JSON Datastore which would be updated every 24 Hours or when the user next launched the tool.
2. The JSON datastore was upgraded to a local sqlite database for ease of querying and potential for multithreading which a simple json file does not support. This solution was tested with both the NIST NVD Database Api and the CVEDetails Database Api
3. The code scanner was spiked as a wrapper for the widely used tool semgrep for full static analysis.

Ultimately the outcomes of a local vulnerability database were less than favourable. The logic required for matching packages against the data provided by the API's tested was cumbersome. The data provided by the external databases was seemingly incomplete. This is when the OVS Database was discovered which would allow NAS to parse and transform package.json data into another shape and query in batch to the OVS Database api for a complete understanding of the vulnerabilities present.

The code scanner semgrep spikes were ruled not viable for the NAS use case because it meant that the project would either have to bundle the semgrep binary statically into the application or initialize the semgrep cli natively on the system. For the NAS project team this was too heavy of a dependency and too large of a risk. This resulted in the NAS team implementing a module with regex pattern matching for SAST with both generally vulnerable patterns in web development and Next.js specific patterns.

2.2.4. Implementation Overview

This tool has two possible configuration interfaces, two end-to-end functions and one output. The user can interface via configuration of a config.yaml file specified in an initialization argument, or they can choose to specify everything in the command line interface during the initialization argument. The first function is the Next.js specific code scanner we have created with no external calls or integration to third-parties and the second is the dependency scanner we have also created which leverages an external api to alleviate the necessity to aggregate vulnerability intelligence. The result of these two functions is output as a pdf which summarizes the project then dives into the detail of threat identified individually. The code scanner shows the file, line and the exact code which is vulnerable. The details provided by the dependency scanner are: title, a summary, detailed explanation of the threat, patches, and if available commands to run for remediation, workarounds, timeline and references to the sources of documentation. The Code Scanner matches for regex patterns we assert are vulnerable based on [5]. The dependency scanner leverages the existing "Open Source Vulnerability (OSV) Database" API to leverage the 24 sources they aggregate threat intelligence from [4].

2.2.5. Component Details

CLI The aim of the CLI is to handle two forms of input specification: Config file and CLI arguments, and to then initialize and orchestrate the applications functions based on those user specifications. The CLI then either prints a basic summary of vulnerabilities detected or an in depth explanation, based on the verbosity flag.

Dependency Scanner The aim of the dependency scanner is to scan the list of packages used by the given *next.js* project to find possible known vulnerable packages. Dependency scanning requirements are met by creating a module which both parses and transforms package.json data for information retrieval from the OVS Database API. This decision reduces complexity and removes the onus of maintaining an up-to-date database with vulnerability to package relationships. NAS is using the OSV Database API's based on its simple request and response formats and its ability to return vulnerability ID's from batch lists of packages from varying environments. These ID's are then queried against another OVS Database API Endpoint to retrieve more detailed information including title, summary, detailed explanation of the threat, patches, and if available commands to run for remediation, workarounds, timeline and references to the sources of documentation. NAS parses the packages and sends them in batch form to the API, this decision was made instead of integrating their cli tool directly, this results in the outcomes:

1. Individually querying dependencies and dev dependencies can result in more granular reporting.
2. package.json can be parsed and transformed by us ensuring the correct packages are sent and queried against
3. If a new, better database for querying becomes available in the future the OSV Database API can be swapped

Code Scanner Our code scanning components uses *REGEX* to scan for possible dangerous code part. The following rules are implemented in our scanner:

- **API Keys:** Miss included API Keys or password in the client code of the project.
- **Exposed CORS:**
- **Remote code execution attack surfaces:** By searching for code snippets that include function as `"Eval () "`.
- **Unsafe use of the function constructor:** Generating code dynamically with `Function()` constructors
- **Unsafe use of the javascript virtual machine:** Context manipulation / injection vulnerabilities
- **Hardcoded secrets:** Sensitive credentials stored in the source code as plaintext
- **Cloud provider keys:** Sensitive keys for cloud provider access stored as plaintext in the source code
- **Exposed JWT Secrets:** Leakage of the keys used for generating Javascript Web Tokens
- **Database url's :** Database connection strings with secrets hardcoded
- **Unsafe invocation of child processes:** invocation of `exec()` and `spawn()` with unvalidated and unsanitized user inputs
- **Next.js DangerouslySetInnerHTML prop:** This prop bypasses Next.js (And React) string sanitization preventing SQL injection
- **React & Next.js design patterns which negate built in security mechanisms dangerously:**
 - Setting:
 - * `InnterHTML`
 - * `OuterHTML`
 - * `AdjacentHTML`
 - * Unsafe content in Next.js head component

All of the above patterns are direct DOM manipulation which bypasses React & Next.js XSS Protection
 - Bypassing:
 - * **Next.js Script sanitization:** This negates any script loading sanitization and protections built into Next.js
 - * **Server-side rendering sanitization (SSR):** Next.js has special safeguards for manipulating SSR for XSS Methods
- **Importing based on user input:** Dynamic imports which rely on user-controlled paths that are unvalidated
- **Placing event handlers inline:** user controlled content within an inline event handler
- **Placing Javascript URL's in href & src attributes:** link and source attributes containing Javascript protocol URLs
- **Dynamically placing content in redirect url's from Next.js middleware:** redirect destinations which are totally unvalidated originating from middleware functions

PDF Generator The PDF Generators function is to take the results struct depicted in 2 and transform that from data into a professional report tailored to the project scanned. This report containing information that can be leveraged by a developer, engineer, or security analyst.

3. Results

We performed the experiments in a controlled environment designed to simulate a real-use scenario. All test was performed in *Debian 12* operating system with *go 1.24.3*. An internet connection is required to perform the dependency scan.

A control repository was forked by the NAS team ¹ which included many know Next.js vulnerabilities relating to the application fullstack including now-patched middleware vulnerabilities [7]. This repository was then injected with the vulnerabilities which should successfully trigger the NAS Code Scanner SAST. After successfully completing end-to-end testing on this baseline repository attention was focused on targeting public repositories on Github.

The experiments are done on a combination of a well-known vulnerable Next.js project and a project randomly chosen from the open-source Next.js community. In our test, both code scan and dependency scan were enabled.

The source code of our tool and our test dataset is publicly available in our repository ² to facilitate the reproducibility of our experiments.

3.1. Vulnerable Next.js Project Analysis

The tool used code and dependency scanning together on a Strobe Security Vulnerable Next.js application and it discovered 15. The vulnerability breakdown indicated 13 vulnerability dependencies (86.67%) and 2 code vulnerabilities (13.33%) meaning that the scanner was productive in dual-mode scanning which captures both internal and external sources of security risk when scanning. The 2 code vulnerabilities identified by the tool were databases exposures which are a serious issue in any modern web application.

3.2. Community Next.js Project Analysis

Analyzing randomly selected open-source Next.js community projects identified similar observable results as with the Strobe vulnerable application. The total vulnerability of community projects was 163, of which 153 were dependency issues and only 10 code scan. A sizeable number of projects had no vulnerabilities identified, which suggest code quality among the community is relatively high, but package management for security reasons has to improve.

4. Discussion

This section contains a complete discussion of the experimental results which were accumulative in that they are derived from scanning 50 different Next.js repositories that encompassed a variety of project types and complexities. The discussion was reviewed thoroughly, with the intent

¹<https://git.mif.vu.lt/micac/2025/nextjs-vuln>

²<https://git.mif.vu.lt/micac/2025/attack-surface>

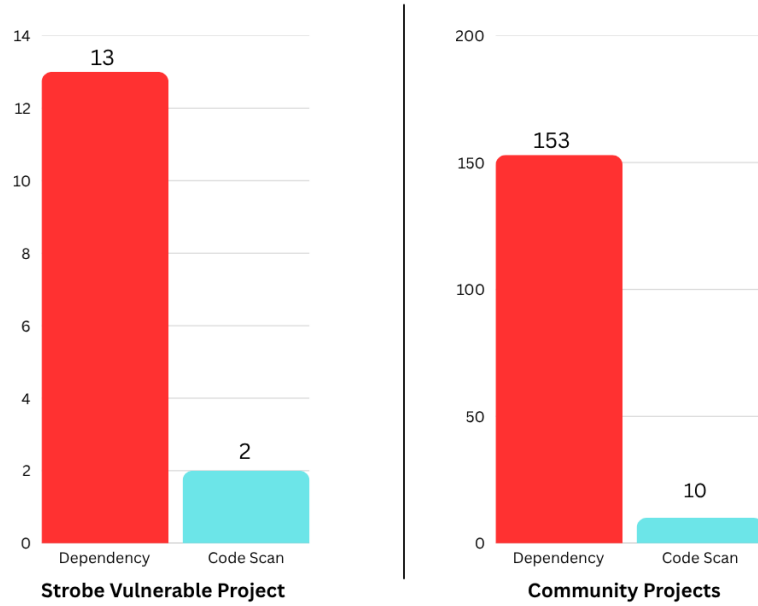


Figure 3: Vulnerability Distribution Analysis

to discuss this overall experimental findings and implications first, and then discuss the limitations of the analysis strategy, and opportunities for ongoing improvements.

4.1. Experiment Evaluation

The findings of the comprehensive assessment of 50 repositories indicated important details about the scanner’s performance characteristics and its ability to detect problems. The findings show both the potential and obstacles that exist in identifying vulnerabilities automatically in modern web applications. Based on the assessment, we are identifying the overwhelming presence of dependency vulnerabilities, representing 93.3% (166 of 178) of all vulnerabilities for the community projects we analyzed. This distinct distribution has consequences that essentially overturn established assumptions about web application security priorities while providing important evidence about the contemporary JavaScript development ecosystem. We believe that the 13.8:1 dependency-to-code vulnerability ratio indicates that supply chain security issues are a major issue for NextJS applications, rather than by logic-related or code quality issues. This finding encapsulates a significant part of recent industry reports, which describe the increasingly sophisticated nature of supply chain attacks. The fact that our vulnerability scanner is able to identify issues related to dependency vulnerabilities quite easily is further evidenced by the fact that we were able to identify a vulnerable package in all 50 projects we analyzed without fail (100% detection rate). This should give practical reasons for believing that the vulnerability scanner can be successfully used as a specialized assessment tool for NextJS security.

We also found 15 vulnerabilities, 13 dependency and 2 code, in an intentionally vulnerable Strobes repository. While confirming that it detected vulnerabilities accurately in a semi-controlled environment, it also reflected the vulnerability distributions we observed in community projects, and confirms confidence in the detection capabilities for synthetic and authentic conditions of vulnerability. The effectiveness observed with the scanner substantiates the argument for framework-specific security tooling, rather than just targeting generic, all-purpose vulnerability scanner solu-

tions, in today's web development practices. While a vulnerability scanner targeting vulnerabilities generically may look at a range of libraries or dependencies being included in a web application, they are unable to determine the underlying nuances, such as the dependency relationship of all the dependencies in the framework, and any security patterns specific to that framework that the scanner can take advantage of. The results presented here provide evidence for investing in framework-specific development tooling for security and development teams, within common and popular frameworks, in this case NextJS.

The analysis of the execution time has indicated a significant degree of variability, with scans taking as little as 0.133 seconds, and as long as 94.349 seconds. Since the variability in execution time was nearly 700 times, it suggests that the performance of the scanner is highly dependent on project complexity and structure, rather than the size of the codebase. The overwhelming majority of successful scans (70%) were completed in 10 seconds or less, indicates reasonable performance for most use cases. Memory usage values, on the other hand, illustrate reasonable consumption of resources (0.008MB to 0.96MB), and suggest that the underlying scanner is somewhat performant when it is functioning well.

4.2. Limitations

There are some limitations based on the current architecture of our tool that limit its value and impact. For example, the dependency vulnerability detection component relies entirely on vulnerability databases and matching techniques that do not capture the quintessential ways in which Next.js applications adopt dependencies and configure them and other elements like multiple build-time optimizations and runtime configurations that greatly affect the actual exploitation of dependency vulnerabilities, which all attention to the subtleties in particular static scanning methods.

The lack of structural comparison including other NextJS security tools or general-purpose vulnerability scanners makes it difficult for us to identify performance benchmarks other than our own. Analysis of the application alone, comparing it to another tool, would help provide some context to identify its performance advantages or limitations with respect to existing alternatives. By extension, we have no way to make conclusions about the value proposition of the scanner in its current form versus the capabilities provided by existing tools when considered together.

Our scanner uses static code analysis, and therefore is limited in its ability to find runtime security issues, configuration vulnerabilities, or deployment-type security issues. Many of the threats to modern web applications only come to light at times of execution, which cannot be identified by static analysis tools and this limitation can makes it hard to properly test NextJS security, as a lot of client-side processes are managed by NextJS.

As it is currently constructed the scanner does not either do severity scoring or provide risk prioritization options for the vulnerabilities it detects. This means that all vulnerabilities have the same weight from our detection regardless of actual researchers' potential impact and exploitability. This can create alert fatigue and reduction in developer adoption for security teams since they can be limited by the sheer volume of findings of varying genuine importance. Our tool also does not have any integrations with commonly used development environments or security orchestration platforms which limits its ability to be used in a complete security workflow.

Conclusions and Future Work

We have built a Next.js vulnerability scanner that has significant technical value as a security analysis tool with particular value as a dedicated tool for security analysis, as evidenced by robust supporting architecture and consistent application operational characteristics that will support its use in serious professional development. The scanner meets an important need for security analysis in custom Next.js applications, while also generally suitable performance characteristics for use in a continuous integration pipeline to automate security analysis. The holistic strategy we took with the combined code and dependency analysis gives our scanner considerable new value as part of the security tool chain for Next.js applications.

Our development plan is to improve the accuracy of the scanner by applying machine learning algorithms that can learn from user patterns and improve over time as well as reduce false positive rates. We will further develop a contextual analysis module to understand the Next.js patterns and conventions better, which will greatly help improve the context of the vulnerabilities raised. We will also explore developing sophisticated risk scoring algorithms and prioritization to rate vulnerabilities based on exploitability, impact and environmental context, to also improve the practical use of the scanner for security teams managing large application portfolios.

In terms of our roadmap for research, we are exploring the development of advanced static analysis techniques and tools such as, taint analysis, and symbolic execution techniques as they could greatly improve the scanner's ability to detect complex vulnerability patterns as well minimise the detected false positive counts. We are also considering the use of dynamic code analysis and hybrid analysis to capture runtime security behaviours which are specific to Next.js applications. Finally, we will aim to have comprehensive reporting and analytics to enable security teams to view vulnerability trends, impact of their remediation activities and to communicate the maturity of the organization's security programs to the appropriate stakeholders.

As NextJS and similar frameworks mature and are being adopted in enterprise settings, the security problems noted in this research will only continue to grow. The tools, methodologies, and insights that were created as part of this work provide a starting place to tackle the problems and elevate the security posture of modern web applications. Moving forward will require continued investment in supply chain security research, tool creation, and education for the community. The quantitative evidence in this study should be a motivating factor to bridge the gap between the real threat landscape and the security practices applied today, so that we can all have more secure and resilient web applications that benefit the broader NextJS community.

References

- [1] Terzi Anastasia and Bibi Stamatia. Managing security vulnerabilities introduced by dependencies in react.js javascript framework. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, pages 126–133, 2024. doi: 10.1109/SANER-C62648.2024.00022.
- [2] Tiago Brito, Mafalda Ferreira, Miguel Monteiro, Pedro Lopes, Miguel Barros, José Fragoso Santos, and Nuno Santos. Study of javascript static analysis tools for vulnerability detection in node.js packages. *IEEE Transactions on Reliability*, 72(4):1324–1339, 2023. doi: 10.1109/TR.2023.3286301.
- [3] Alexandre Decan, Tom Mens, and Eleni Constantinou. On the impact of security vulnerabilities in the npm package dependency network. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR ’18*, page 181–191, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450357166. doi: 10.1145/3196398.3196401. URL <https://doi.org/10.1145/3196398.3196401>.
- [4] Google OSV.dev Team. Data sources. URL <https://google.github.io/osv.dev/data/>. Documentation for vulnerability data sources and ecosystems.
- [5] Edvin Hellquist. Evaluating security for javascript-based frontend frameworks. Master’s thesis, Umeå University, Department of Computing Science, 2025.
- [6] Tobias Lauinger, Abdelberi Chaabane, Sajjad Arshad, William Robertson, Christo Wilson, and Engin Kirda. Thou shalt not depend on me: Analysing the use of outdated javascript libraries on the web. In *Proceedings 2017 Network and Distributed System Security Symposium, NDSS 2017*. Internet Society, 2017. doi: 10.14722/ndss.2017.23414. URL <http://dx.doi.org/10.14722/ndss.2017.23414>.
- [7] NIST - National Vulnerability Database. Cve-2025-29927. URL <https://nvd.nist.gov/vuln/detail/CVE-2025-29927>.